Surface Text Features Do Not Beat the Majority Class for Predicting Robust Reasoning: A Negative Result

Daris Dzakwan Hoesien
Birru Technologies, Indonesia
Codabench user: `darisdzakwan`; submissions 918014 (Main), 918015 (Small)

Abstract

The AIMO Interpretability Challenge asks, for each (model, problem) pair, whether a reasoning model's performance survives reasoning-preserving perturbations (robust) or does not (spurious). About a quarter of the public labels are robust, so predicting "spurious" for everything scores 0.745 on them. We shipped exactly that predictor in both tracks; on the Codabench competition phase it scored only 0.48 (Main) and 0.47 (Small), so a floor measured on the public labels did not transfer. We report how far cheap, model-free features get towards beating the public-label floor. Twenty-two surface features of the problem text, fed to a logistic regression or a gradient-boosted classifier, carry a small but statistically distinguishable ranking signal (AUC 0.65–0.66, permutation $p \leq 0.005$ on 141 problems from one model), but not enough to beat the majority class in accuracy: with the decision threshold chosen inside each training fold, accuracy is 0.70–0.71 against 0.745. The usual protocol of choosing the threshold on the cross-validated predictions is 1–2 points more optimistic.

1 Task and what we submitted

The harness calls `are_robust(model_id, problems)` once per model and expects a list of booleans of exactly the right length; any exception, wrong length or non-boolean element invalidates that model's whole batch. The metric is accuracy over all labels, so coverage must be 1.0. The two tracks differ in the model set (the Small track leaves out `gpt-oss-120b`) and in a root-level `small.txt` marker. A label is positive (robust) iff the model's absolute accuracy decay is zero. The permutation type is not given at evaluation time, only the problem string.

Submission. Both tracks received the same code. `solution.py` wraps every call in a try/except that returns `[False]*n`, and delegates to a predictor that loads a bundled classifier only if the artifact is flagged `beats_majority`. The flag is false (Section 3), so the shipped predictor returns False for every problem. It is a labelled floor, not a model of robustness. Codabench submission 918014 (Main, uploaded 2026-09-07 22:50) and 918015 (Small, 22:51; adds an empty `small.txt`) are the local files `solution-tier0-main.zip` (SHA-256 `8f112222...`) and `solution-tier0-small.zip` (`5060d847...`); we infer this from the file times, and a later rebuild with identical contents but different bytes is not what was uploaded. Local checks (organizers' `run_local.py`, both tracks): accuracy 0.7372, coverage 1.0, 0 invalid predictions on the bundled sample; the contract tests pass; `are_robust` returned a correct-length list of booleans on empty input, one item, a 100,000-character item, control characters, 1,000 items, non-string items and a `None` model id. The Codabench submission list shows both as finished, with scores 0.48 (Main) and 0.47 (Small). That list view reports a single score column and does not itself state whether it is accuracy at coverage 1.0 or break out coverage separately, and it does not show a per-track rank; both would require opening each submission's detailed-results page on Codabench, which has not

been done for this draft. Everywhere else in this report, "score" for these two submissions means exactly this list-view number, not a verified, coverage-adjusted accuracy.

2 Data and method

Data. The public labels (`math-robust-final.csv`, `OVERALL` rows) cover 141 problems, 36 of them robust (25.5%), all for one model (`qwen3-8b:low`) on Hendrycks MATH problems; the private test uses olympiad problems and several other models. The all-False predictor therefore scores $105/141 = 0.745$ on these labels (bootstrap 95% interval $[0.674, 0.816]$). On the Codabench competition phase the same predictor scored 0.48 and 0.47, which would mean that roughly half of the labels there are robust rather than a quarter: the public file is not representative of the platform's labels (different model set, different problems). *The 137-case validation sample used by the local harness contains the same problems as the training file* (exact text overlap 137 of 137), so the local score of 0.7372 $(101/137)$ confirms packaging only; it says nothing about generalisation of a trained classifier.

Features and classifiers. 22 surface features of the problem statement (length, digit and LaTeX counts, keyword flags, sentence statistics), computed without running a language model. A class-balanced logistic regression (standardised inputs, $C = 0.3$) and a depth-3 histogram gradient-boosted classifier (200 iterations, learning rate 0.05, $\ell_2 = 1$, minimum leaf 15). With one row per problem, grouping folds by problem is moot here.

Evaluation. (i) *Nested threshold:* repeated stratified 5-fold CV over 20 seeds, where the decision threshold maximising accuracy is picked on out-of-fold predictions of the training part only. (ii) *Ranking signal:* AUC of out-of-fold probabilities, with a 2,000-resample bootstrap interval and a 1,000-permutation label test. (iii) The accuracy that the training script itself reports, with the threshold chosen on the same cross-validated predictions, for comparison.

3 Results

Predictor	Acc., nested threshold	AUC [95% CI]	Permutation p
All-False (shipped)	0.745 [0.674, 0.816]	0.5	—
Logistic regression	0.702 (sd 0.018)	0.650 [0.545, 0.750]	0.005
Gradient boosting	0.710 (sd 0.018)	0.664 [0.565, 0.753]	0.003

Table 1: Accuracy and ranking signal on the 141 public problems. Accuracy is the mean over 20 CV seeds with the threshold chosen inside each training fold; "sd" is the standard deviation over seeds. The shipped predictor equals the first row.

With the threshold chosen on the same cross-validated predictions the training script reports 0.716 (logistic regression) and 0.731 (gradient boosting): this protocol is 1–2 points more optimistic than the nested one and still does not exceed 0.745. Both classifiers' accuracies fall below the baseline's point estimate but inside its interval, so the honest statement is "not better", not "worse". The non-zero AUC together with no accuracy gain is what one expects at 25% prevalence: a modest ranking signal needs a high decision threshold, and an accuracy-optimal threshold collapses towards the majority prediction. The threshold, not the signal, is the limit. A ship rule requiring cross-validated accuracy at least 0.02 above the majority class is therefore never met, and the classifier stays off.

What the data can resolve. With 141 rows the binomial standard error of an accuracy near 0.745 is about 0.037, so a true gain of two or three points would not be detected. The negative result is a statement about these features at this sample size, not a proof that no text-only method can win. An exact one-sided binomial test of "accuracy ≤ 0.745" on 141 problems needs at least 114 correct (80.9%) to reject at the 5% level; a classifier whose true accuracy is higher by 2, 3, 5, 8 or 10 points passes that test with probability 0.13, 0.20, 0.39, 0.74 and 0.90 respectively. So only gains of about eight points or more are reliably visible at this sample size, and the three-to-four-point gain a weak signal could plausibly give is not.

Where the signal sits. Table 2 gives the out-of-fold AUC (mean over 20 CV seeds) for each feature group alone and with the group removed. For the linear model the signal is in problem size and numeric content: length features alone reach 0.72 and numbers-and-operators alone 0.72, higher than all 22 features together (0.66), while the LaTeX/case and keyword groups alone are near chance (0.51, 0.54) and removing them raises the AUC. The boosted trees also use the LaTeX/case group (0.68 alone), and removing any single group moves their AUC by less than 0.01, so the features are redundant with each other. The keyword flags ("prove", "find", ...) carry almost nothing. These runs used scikit-learn 1.7.2, not the 1.8.0 pin, and the groups are correlated, so the numbers show where the signal *can* come from, not an additive decomposition.

Feature group	LR alone	LR without	GB alone	GB without
all 22 features		0.663		0.667
length (5)	0.723	0.636	0.649	0.664
numbers and operators (6)	0.718	0.655	0.656	0.656
LaTeX and case (5)	0.509	0.692	0.676	0.661
keyword flags (6)	0.539	0.700	0.569	0.668

Table 2: Out-of-fold AUC, mean over 20 seeds (sd 0.01–0.03), 141 problems. LR: logistic regression, GB: gradient boosting; feature counts in parentheses.

4 Limitations and what was not tested

- One model, one problem source, 141 rows; every interval above is wide. The labels come from a single offline perturbation experiment, not from a ground-truth notion of robustness.
- No test on olympiad problems or on the other models. The 25% robust rate of the public file did not hold on the platform: our all-False submissions scored 0.48 and 0.47, so the "0.74 floor" we planned around was an artefact of one model on easy problems, and a constant "robust" predictor (the organizers' always-true baseline) would have scored about 0.52 on the same labels (the complement, assuming coverage 1.0). Every claim about majority-class floors in this report is therefore specific to the public file.
- No competition model was ever run: the development machine had 8 GiB of RAM, no GPU and no Hugging Face token for the private validation sample. Hidden-state probes, token-distribution uncertainty and self-consistency over generated variants, the signals the challenge is built around, are untested by us.
- The organizers' uncertainty-profiling baseline (grouped out-of-fold balanced accuracy 0.626, plain accuracy 0.688, from their task documentation) is cited, not reproduced.
- Pickled classifier artifacts were written with scikit-learn 1.8.0 and numpy 2.x and do not load under older versions; the solution then falls back to all-False, which is also what the shipped predictor does.

5 Reproducibility, data and disclosure

Open-source implementation: `https://github.com/darisdzakwanhoesien2/2026_aimo_interpretability`, commit `4da9dae342c18448dd661aa15da288e1daeffaef`. This is a fresh snapshot (no history carried over) of this challenge entry only, published to satisfy the organizers' open-source requirement for award consideration. Development happens on branch `feat/simulacrabench-aimo` of a separate, private repository (`darisdzakwanhoesien2/research_collaboration`, folder `shared_task_aimo_interp_2026`), which bundles several unrelated research projects and is not itself public; submission sources are in `submission/`, statistics in `experiments/tier0_stats.py` (run with scikit-learn 1.8.0), build record in `kit/BUILD_RECORD.md` — all mirrored into the public repository above. Environment for the checks: Python 3.12, scikit-learn 1.8.0, joblib 1.5.3, numpy 2.5.4. Training data: the organizers' `math-robust-final.csv` only; no external data and no pre-trained model.